

CSG — Mathematischer Deep Dive

Folgeseiten (Entwurf) · Moritz Potthoff

Moritz Potthoff

2026-06-17

Kontext

Was wir bereits wissen

- **SDF:** $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ — negativ innen, null auf der Fläche, positiv außen
- **Ray Marching:** entlang des Strahls mit Schrittweite $|f(\mathbf{p})|$ zur Oberfläche
- **Primitive:** Kugel, Box, Zylinder, ... als geschlossene Formeln

Diese Folien: Wie wird aus Primitiven eine Szene — rein algebraisch?

Von Mengen zu Distanzen

Boolesche Geometrie

Komplexe Körper als Kombination von Grundkörpern $A, B, C, \dots$:

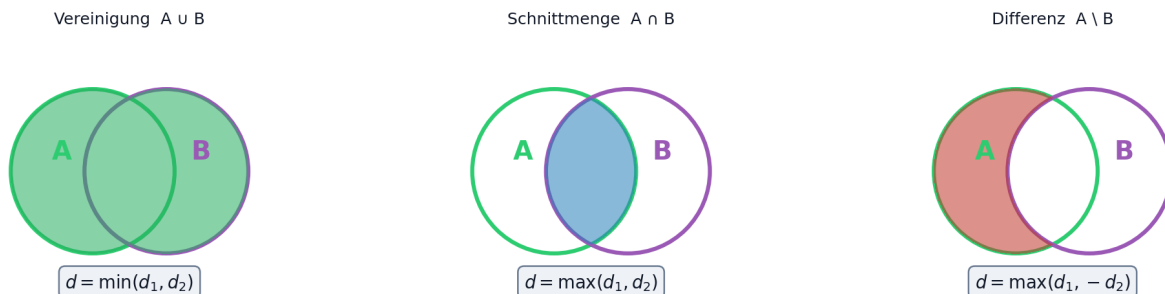
Operation	Mengenschreibweise	Intuition
Vereinigung	$A \cup B$	alles, was in A oder B liegt
Schnittmenge	$A \cap B$	nur der gemeinsame Bereich
Differenz	$A \setminus B$	A ohne B

Bei **Meshes** werden Dreiecke geschnitten — teuer und fehleranfällig.

Bei **SDFs** reicht eine Kombination der Distanzwerte pro Punkt $\mathbf{p}$.

Venn-Diagramme → SDF-Formeln

Boolesche Operationen → SDF-Kombination



Die drei Kernoperationen

SDF-Funktionen

$$d_{\text{Kugel}}(\mathbf{p}) = \|\mathbf{p} - \mathbf{c}\| - r$$

$$\mathbf{q}_{\text{Box}} = |\mathbf{p} - \mathbf{c}| - \mathbf{b}$$

$$d_{\text{Box}}(\mathbf{p}) = \|\max(\mathbf{q}_{\text{Box}}, \mathbf{0})\| + \min(\max(q_x, q_y, q_z), 0)$$

$$\mathbf{q}_{\text{Zyl}} = \left(\sqrt{(p_x - c_x)^2 + (p_z - c_z)^2}, |p_y - c_y| \right) - (r, h)$$

$$d_{\text{Zylinder}}(\mathbf{p}) = \|\max(\mathbf{q}_{\text{Zyl}}, \mathbf{0})\| + \min(\max(q_x, q_y), 0)$$

$$\mathbf{q}_{\text{Torus}} = \left(\sqrt{(p_x - c_x)^2 + (p_z - c_z)^2} - R, p_y - c_y \right)$$

$$d_{\text{Torus}}(\mathbf{p}) = \|\mathbf{q}_{\text{Torus}}\| - r$$

$$d_{A \cup B}(\mathbf{p}) = \min(d_A(\mathbf{p}), d_B(\mathbf{p}))$$

$$d_{A \cap B}(\mathbf{p}) = \max(d_A(\mathbf{p}), d_B(\mathbf{p}))$$

$$d_{A \setminus B}(\mathbf{p}) = \max(d_A(\mathbf{p}), -d_B(\mathbf{p}))$$

Boolesche Funktionen

$$\mathbf{p} \in A \iff d_A(\mathbf{p}) < 0$$

$$\mathbf{p} \in A \cup B \iff (d_A(\mathbf{p}) < 0) \vee (d_B(\mathbf{p}) < 0) \iff \min(d_A, d_B) < 0$$

$$\mathbf{p} \in A \cap B \iff (d_A(\mathbf{p}) < 0) \wedge (d_B(\mathbf{p}) < 0) \iff \max(d_A, d_B) < 0$$

$$\mathbf{p} \in A \setminus B \iff (d_A(\mathbf{p}) < 0) \wedge \neg(d_B(\mathbf{p}) < 0) \iff \max(d_A, -d_B) < 0$$

$$f_{\cup}(d_A, d_B) = \min(d_A, d_B)$$

$$f_{\cap}(d_A, d_B) = \max(d_A, d_B)$$

$$f_{\setminus}(d_A, d_B) = \max(d_A, -d_B)$$

Algebraische Definition

Für zwei SDFs $d_1(\mathbf{p})$, $d_2(\mathbf{p})$:

$$\text{Union: } d(\mathbf{p}) = \min(d_1(\mathbf{p}), d_2(\mathbf{p}))$$

$$\text{Intersection: } d(\mathbf{p}) = \max(d_1(\mathbf{p}), d_2(\mathbf{p}))$$

$$\text{Difference: } d(\mathbf{p}) = \max(d_1(\mathbf{p}), -d_2(\mathbf{p}))$$

Differenz-Trick: $-d_2$ spiegelt innen/außen von B — das „Herausschneiden“ wird zu einem Maximum.

GLSL-Implementierung

```
float opUnion(float d1, float d2) { return min(d1, d2); }  
float opIntersect(float d1, float d2) { return max(d1, d2); }  
float opSubtract(float d1, float d2) { return max(d1, -d2); }
```

Die gesamte Szene ist eine **verschachtelte Ausdrucksbaum-Auswertung** — kein neues Mesh, keine Topologie-Änderung zur Laufzeit.

Warum funktioniert das?

Innen/Außen-Logik

Ein Punkt liegt **innen**, wenn $d < 0$ (genug negativ = tief im Körper).

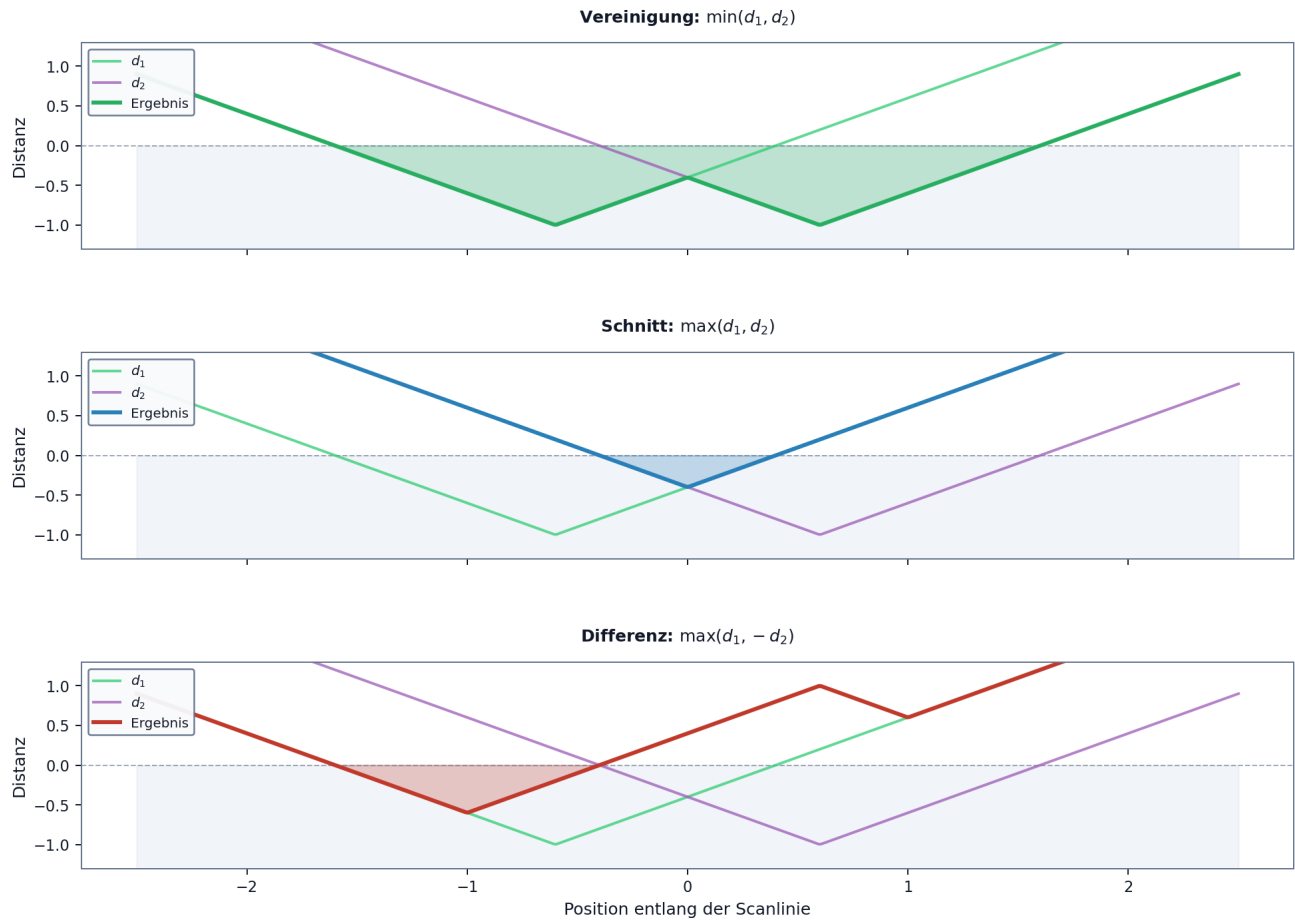
Warum min / max? — Innen iff alle relevanten $d_i < 0$

Punktlage	d_1	d_2	Union min	Schnitt max	Diff max(·, -)
nur in A	< 0	> 0	$< 0 \checkmark$	$> 0 \times$	$< 0 \checkmark$
nur in B	> 0	< 0	$< 0 \checkmark$	$> 0 \times$	$> 0 \times$
in $A \cap B$	< 0	< 0	$< 0 \checkmark$	$< 0 \checkmark$	$< 0 \checkmark$
außen	> 0	> 0	$> 0 \times$	$> 0 \times$	$> 0 \times$

- **Union:** min wählt die *nähere* Fläche $\rightarrow$ innen, wenn *mindestens eine* Form innen ist
- **Schnitt:** max verlangt, dass *beide* Distanzen negativ sind
- **Differenz:** $-d_2$ kehrt B um — innen in A , aber außerhalb von B

1D-Scanline: min / max sichtbar machen

1D-Intuition: horizontale Probe durch zwei Kreise

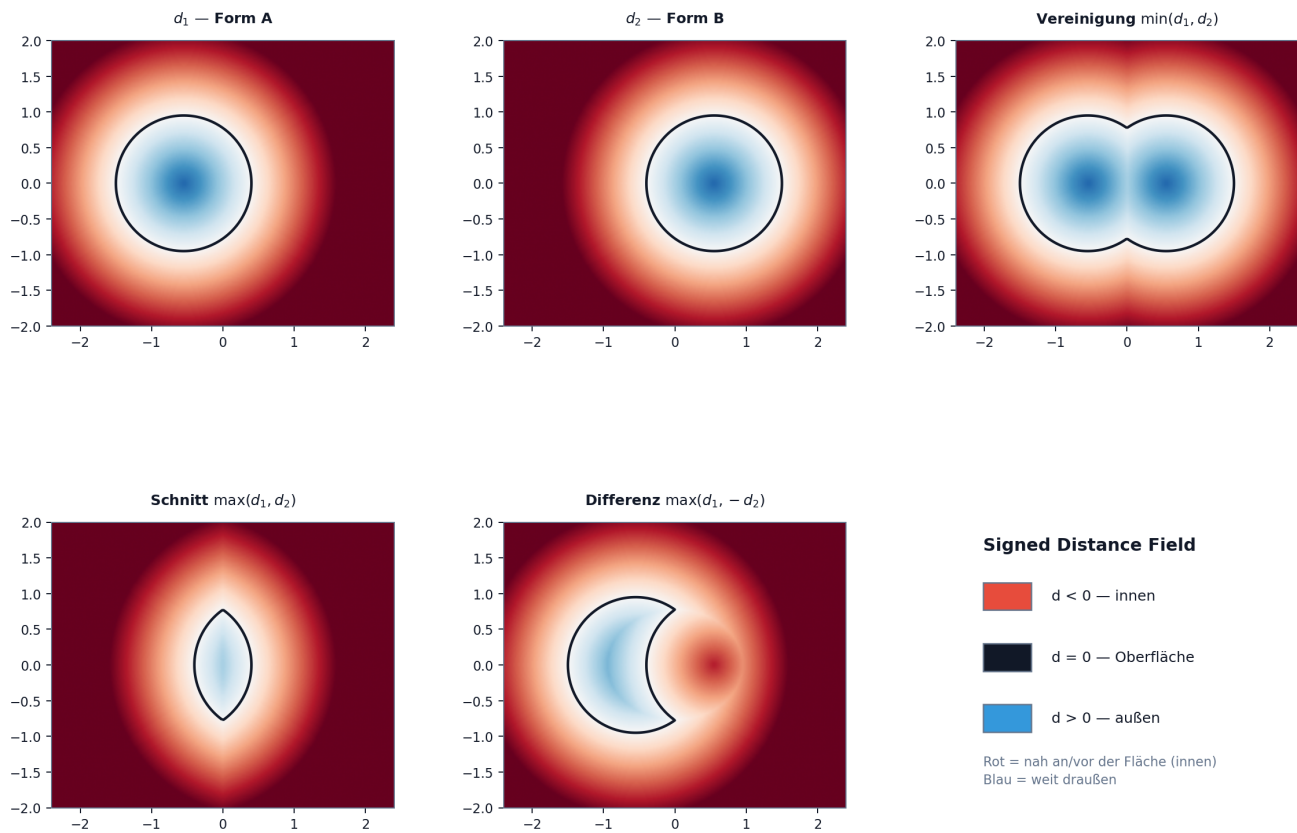


Horizontale Probe durch zwei Kreise: d_1 , d_2 und das kombinierte Feld. Rot schattiert = $d \leq 0$ (innen).

Visuell im 2D-Feld

Distanzfelder vor und nach CSG

2D-Querschnitt: Distanzfelder vor und nach CSG



Weißer Kontur = Nullniveau ($d = 0$) = Oberfläche. Gleiche Operationen funktionieren in 3D — nur die Formeln der Primitive ändern sich.

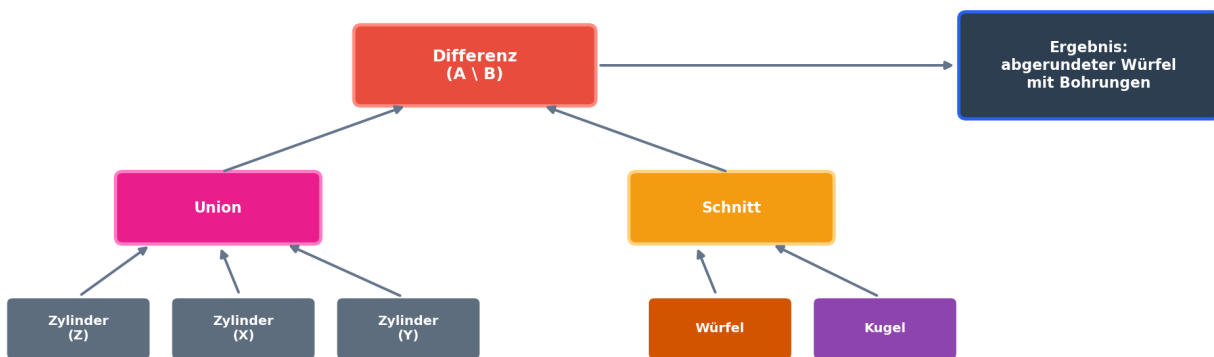
CSG als Baum

Hierarchische Konstruktion

CSG-Baum: hierarchische Konstruktion

Blätter = Primitive SDFs · Knoten = boolesche Operatoren · Wurzel = finale Szene-SDF

```
float scene(vec3 p) {  
    float holes = opUnion(opUnion(cylZ(p), cylX(p)), cylY(p));  
    float core = opIntersect(box(p), sphere(p));  
    return opSubtract(core, holes);  
}
```



Blätter = Primitive · innere Knoten = Operatoren · Wurzel = `scene(p)` für Ray Marching

N-äre Operationen

N-äre Vereinigung = wiederholtes min

```
float d = d1;  
d = min(d, d2);  
d = min(d, d3);  
d = min(d, d4);
```



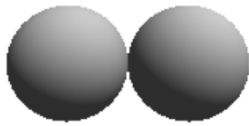
Mehrere Objekte = **assoziatives** Verkettens derselben Operation (Reihenfolge bei Union/Diff egal).

Beispiel: gerenderte Szene

Vom Baum zum Bild

Ray-Marched CSG-Beispiele (aus SDF-Formeln)

Union zweier Kugeln



Schnitt Box $\cap$ Kugel



Differenz Box $\setminus$ Kugel



CSG-Baum: Würfel mit Bohrungen



Software-Raymarch aus denselben Formeln — kein Mesh. Rechts unten: der FreeCAD-ähnliche CSG-Baum.

Vollständiger Shader-Ausschnitt

```
float sdCylinder(vec3 p, float h, float r) { /* ... */ }

float scene(vec3 p) {
    float holes = min(min(sdCylinder(p.zxy, 1.2, 0.22),
                          sdCylinder(p.yzx, 1.2, 0.22)),
                      sdCylinder(p, 1.2, 0.22));
    float core = max(sdBox(p, vec3(0.75)),
                    sdSphere(p, 0.95) - 0.0);
    return max(core, -holes); // Differenz
}
```

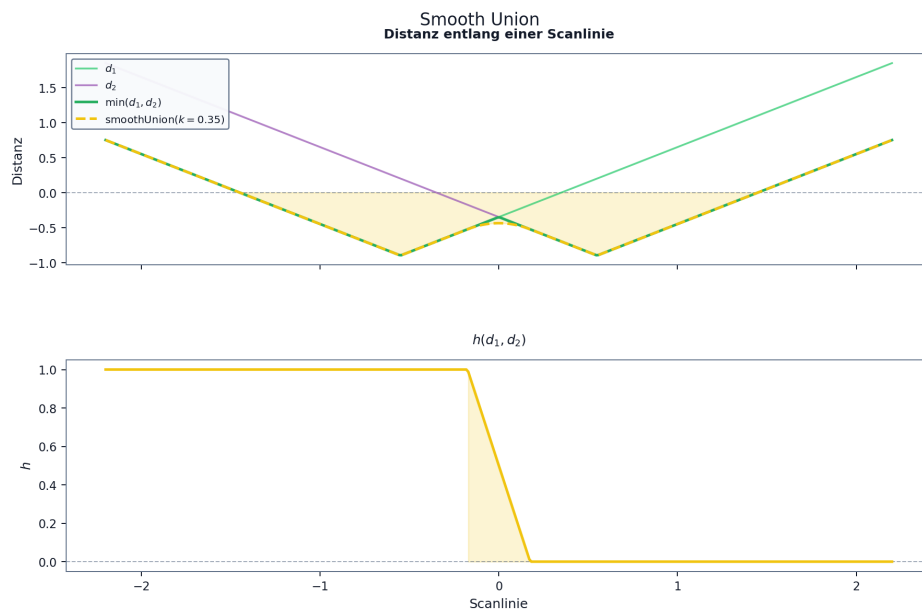
Erweiterungen (kurz)

Was noch mit derselben Idee geht

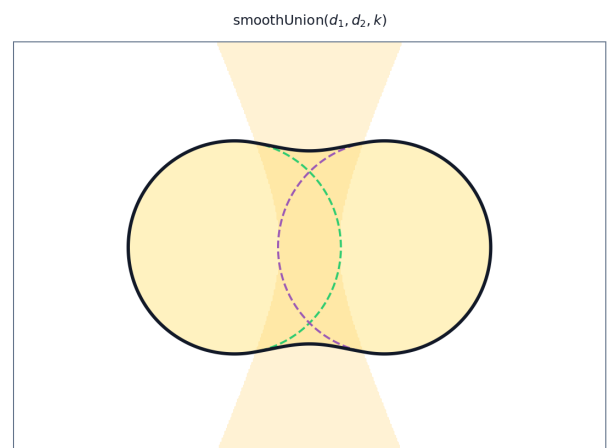
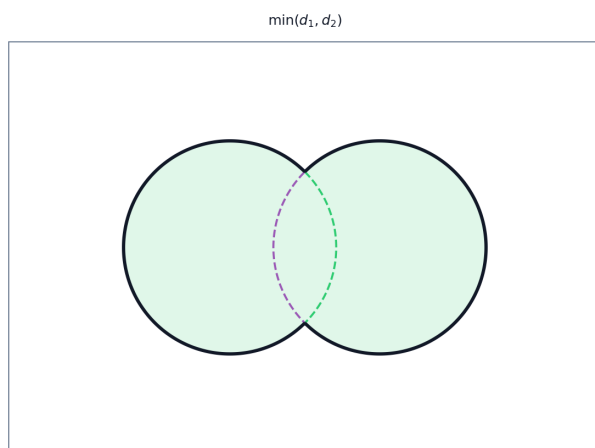
Glatte CSG — scharfe Kanten durch polynomiale Mischung ersetzen:

```
float smoothUnion(float d1, float d2, float k) {
    float h = clamp(0.5 + 0.5*(d2-d1)/k, 0.0, 1.0);
    return mix(d2, d1, h) - k*h*(1.0-h);
}
```

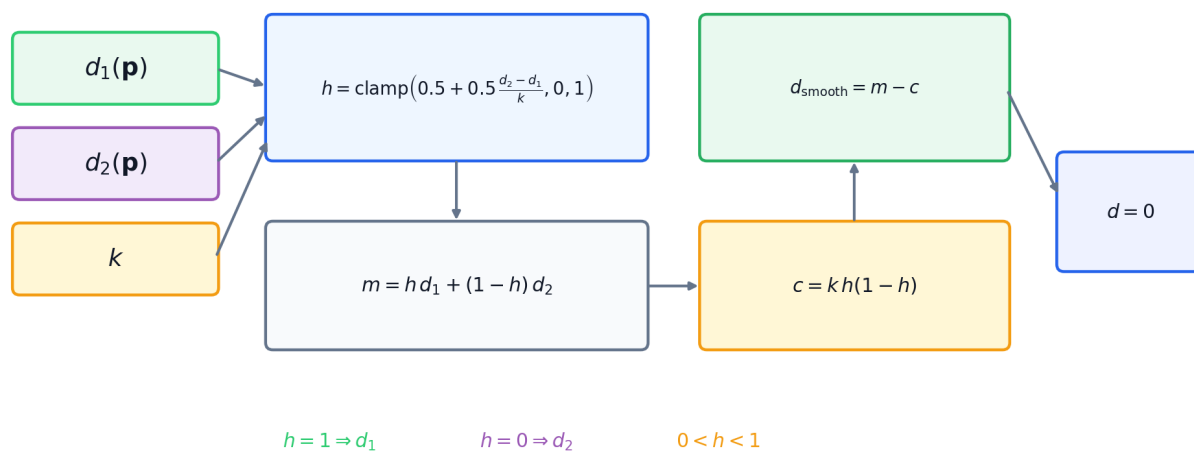
}



Smooth Union: Idee



Smooth Union: Rechnung



Weitere Anwendungen

- **Verschachtelung beliebiger Tiefe** — CSG-Bäume, nicht nur zwei Operanden
- **Domain-Operationen** — Translation, Rotation, Skalierung *vor* der SDF-Auswertung
- **Wiederholung** — `mod(p, spacing)` für Gitter/Tunnel (kein boolesches Op, aber kombinierbar)
- **Performance:** eine Funktion `map(p)` — GPU-freundlich, parallel pro Pixel

Kernbotschaft: CSG auf SDFs = boolesche Mengenalgebra, umgesetzt als punktweise min/max — die Brücke zwischen „Modellierungsbaum“ und „Ray-Marching-Schleife“.

Zusammenfassung

Union	$\min(d_1, d_2)$ — nähere Fläche
Schnitt	$\max(d_1, d_2)$ — beide innen
Differenz	$\max(d_1, -d_2)$ — invertiertes B
Szene	verschachtelter Ausdrucksbaum
Rendering	<code>map(p)</code> in der Ray-Marching-Schleife

Fragen?